

Java Backend Architecture

Interview Series

Chapter 19.3

AWS Core Services for Java Apps

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.3 – AWS Core Services for Java Apps

Overview

AWS provides the dominant cloud platform for Java backend deployments. EC2, ECS, and EKS are the three main compute options for Java services; IAM controls access and security. Understanding instance families, auto-scaling, ECS task definitions, and EKS managed node groups enables informed architectural decisions for Java microservice deployments at scale.

Key Definitions

Term	Definition
EC2	Elastic Compute Cloud. Virtual machines. Instance families: t3 (burstable), m5 (general), c5 (compute), r5 (memory).
Auto Scaling Group (ASG)	Manages EC2 fleet with desired/min/max counts and scaling policies based on CloudWatch metrics.
ECS	Elastic Container Service. AWS-native container orchestration. Task definitions define containers; Services manage replicas.
Fargate	Serverless compute for ECS/EKS. No EC2 management; pay per vCPU+GB/s. Ideal for Java microservices.
EKS	Elastic Kubernetes Service. AWS-managed Kubernetes control plane. Managed node groups (EC2 ASG) or Fargate profiles.
IAM Role	AWS identity with attached policies. EC2 instances and ECS tasks assume roles for service-to-service auth.
Launch Template	Versioned EC2 configuration (AMI, instance type, user data) for ASGs. Enables rolling AMI updates.
IRSA	IAM Roles for Service Accounts. EKS pods assume IAM roles via OIDC token exchange (no credentials in pods).
VPC	Virtual Private Cloud. Isolated network with public subnets (LB), private subnets (Java apps), and NAT Gateway.
ALB	Application Load Balancer. L7 HTTP/HTTPS LB with path-based routing, WAF, and target group health checks.

Diagram 1: AWS Java Architecture (EKS)

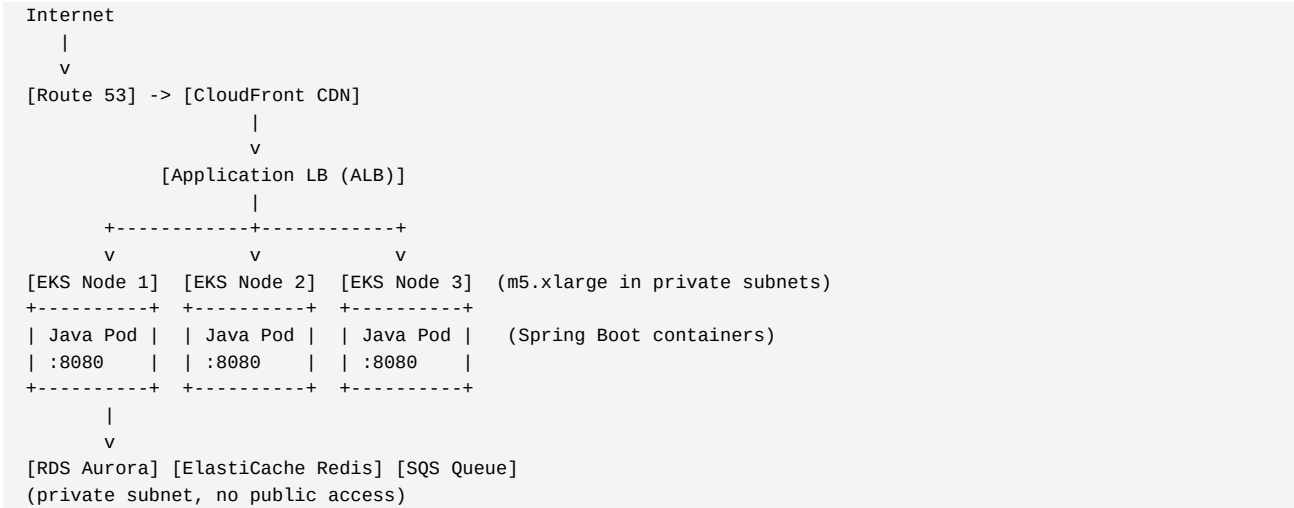


Diagram 2: ECS Task Definition Structure



Code Examples

ECS Service with Fargate:

```
# terraform
resource "aws_ecs_task_definition" "java_api" {
  family           = "java-api"
  network_mode     = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu              = "1024"
  memory           = "2048"
  execution_role_arn = aws_iam_role.ecs_execution.arn
  task_role_arn     = aws_iam_role.java_task.arn

  container_definitions = jsonencode([
    {
      name       = "java-api"
      image      = "${aws_ecr_repository.java_api.repository_url}:latest"
      portMappings = [{ containerPort = 8080 }]
      secrets    = [{ name = "DB_PASSWORD", valueFrom = aws_ssm_parameter.db_pass.arn }]
      logConfiguration = {
        logDriver = "awslogs"
        options = { awslogs-group = "/ecs/java-api", awslogs-region = "us-east-1" }
      }
      healthCheck = {
        command = ["CMD-SHELL", "curl -f http://localhost:8080/actuator/health || exit 1"]
        interval = 30, timeout = 5, retries = 3, startPeriod = 60
      }
    }
  ])
}
```

EKS Managed Node Group (Terraform):

```
resource "aws_eks_cluster" "java_cluster" {
  name     = "java-prod"
  version  = "1.29"
  role_arn = aws_iam_role.eks_cluster.arn
  vpc_config {
    subnet_ids = var.private_subnet_ids
  }
}

resource "aws_eks_node_group" "java_workers" {
  cluster_name = aws_eks_cluster.java_cluster.name
  node_group_name = "java-workers"
  node_role_arn = aws_iam_role.eks_node.arn
  subnet_ids    = var.private_subnet_ids
  instance_types = ["m5.xlarge"]

  scaling_config {
    desired_size = 3; min_size = 3; max_size = 10
  }
  update_config { max_unavailable = 1 } # rolling node update
}
```

IRSA -- Pod IAM Role (no credentials in pods):

```
# Terraform: create IRSA role
data "aws_iam_policy_document" "java_app_assume" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals { type = "Federated"; identifiers = [aws_iam_openid_connect_provider.eks.arn] }
    condition {
      test      = "StringEquals"
      variable  = "${aws_iam_openid_connect_provider.eks.url}:sub"
      values    = ["system:serviceaccount:default:java-app-sa"]
    }
  }
}
```

```
# K8s ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123:role/java-app-s3-role
```

Interview Q&A – AWS Core Services for Java Apps

Q1. Which EC2 instance family is best for Java applications?

m5 (general purpose): best for most Spring Boot apps (balanced CPU/memory). r5 (memory optimised): for memory-intensive Java apps (large heaps, in-memory caching). c5 (compute optimised): for CPU-intensive workloads (data processing, ML serving). t3 (burstable): for dev/staging or low-traffic services (CPU credits for bursts). i3 (storage optimised): for Elasticsearch, Cassandra with local NVMe. EKS: use m5.xlarge (4 vCPU, 16GB) as baseline for Java microservices.

Q2. What is the difference between ECS and EKS?

ECS: AWS-proprietary orchestration. Simpler, tighter AWS integration (IAM, CloudWatch, ALB). No Kubernetes complexity. Good for teams new to containers. Task definitions in JSON. EKS: AWS-managed Kubernetes. Portable workloads (move to GKE/AKS), rich ecosystem (Helm, operators, KEDA). More operational complexity. Better for teams with K8s expertise or complex scheduling requirements. Both support Fargate for serverless compute.

Q3. How does Fargate differ from EC2 launch type in ECS?

EC2 launch type: you manage EC2 instances (patching, capacity planning, scaling). Fargate: AWS manages the underlying compute -- you specify CPU/memory per task, AWS provisions and manages instances transparently. Fargate pros: no EC2 management, per-task billing, automatic security patching. Cons: 20-30s cold start for Java (JVM init), slightly higher cost per vCPU-hour, no EBS volumes. Best for variable-load Java services.

Q4. What is IRSA and why should Java pods use it?

IRSA (IAM Roles for Service Accounts): EKS pods assume IAM roles via Kubernetes Service Account annotations + OIDC token exchange. Pod gets temporary credentials refreshed automatically. Without IRSA: Java apps need long-lived IAM access keys as environment variables (security risk, manual rotation). With IRSA: AWS SDK auto-discovers credentials from EKS metadata -- no credentials in config, pod can be in 10 seconds if compromised.

Q5. How does ASG scaling work with Java services?

ASG tracks CloudWatch metrics: CPU utilisation (target tracking policy: target=70% triggers scale-out), custom metrics (SQS queue depth, ALB request count per target). Scale-out adds EC2 instances (launch template), health check passes, registers with ALB target group. Scale-in: cooldown period (300s default) prevents thrashing. Lifecycle hooks: EC2_INSTANCE_TERMINATING hook lets Java Spring Boot drain connections before instance termination (send graceful shutdown signal).

Q6. What is an ECS Task Role vs Execution Role?

Execution Role: used by ECS agent to pull Docker images from ECR, write logs to CloudWatch, retrieve secrets from SSM/Secrets Manager. Attached to ECS infrastructure. Task Role: assumed by the running Java container to make AWS API calls (read S3, publish to SQS, query DynamoDB). Attached to the Java application. Principle of least privilege: task role should have only the permissions the Java app needs, nothing more.

Q7. How do you configure rolling deployments on ECS?

ECS Service deployment configuration: minimumHealthyPercent=100, maximumPercent=200 (rolling: new tasks started before old stopped, zero downtime). Deployment circuit breaker: rollbackOnFailure=true -- if new task fails health check N times, revert to previous task definition automatically. For Java: set startPeriod in health check to allow JVM warm-up (60-120s) before health checks begin.

Q8. How does the ALB integrate with ECS for Java services?

ECS Service registers task IP:port as ALB target group targets. ALB health checks (HTTP GET /actuator/health) determine target health. Healthy tasks receive traffic; unhealthy tasks removed from rotation. Path-based routing: ALB listener rules route /api/users -> user-service target group, /api/orders -> order-service target group. ECS replaces deregistered targets automatically on deployment.

Q9. What is AWS VPC design for Java microservices?

3-tier VPC: Public subnets (ALB, NAT Gateway -- cross-AZ), Private subnets (Java ECS/EKS tasks -- no direct internet), Database subnets (RDS -- most isolated). Security groups: ALB SG allows 0.0.0.0/0:443; Java task SG allows only ALB SG:8080; RDS SG allows only Java task SG:5432. NAT Gateway for outbound internet from private subnets (ECR pulls, external API calls). Spread across 3 AZs for HA.

Q10. How do EKS add-ons simplify cluster management?

EKS managed add-ons: CoreDNS (cluster DNS), kube-proxy (Service networking), AWS VPC CNI (pod networking with VPC IPs), EBS CSI Driver (PersistentVolumes for Java stateful apps), EFS CSI Driver (ReadWriteMany volumes). Managed by AWS:

automatic version upgrades, conflict resolution with EKS versions. Install via Terraform (aws_eks_addon resource) or eksctl. Critical for Java StatefulSets (Kafka, databases) needing persistent storage.